

Informed Search Algorithms: GBFS & A*

Addis Ababa Sub-City Graph | Fundamentals of AI — Group Project

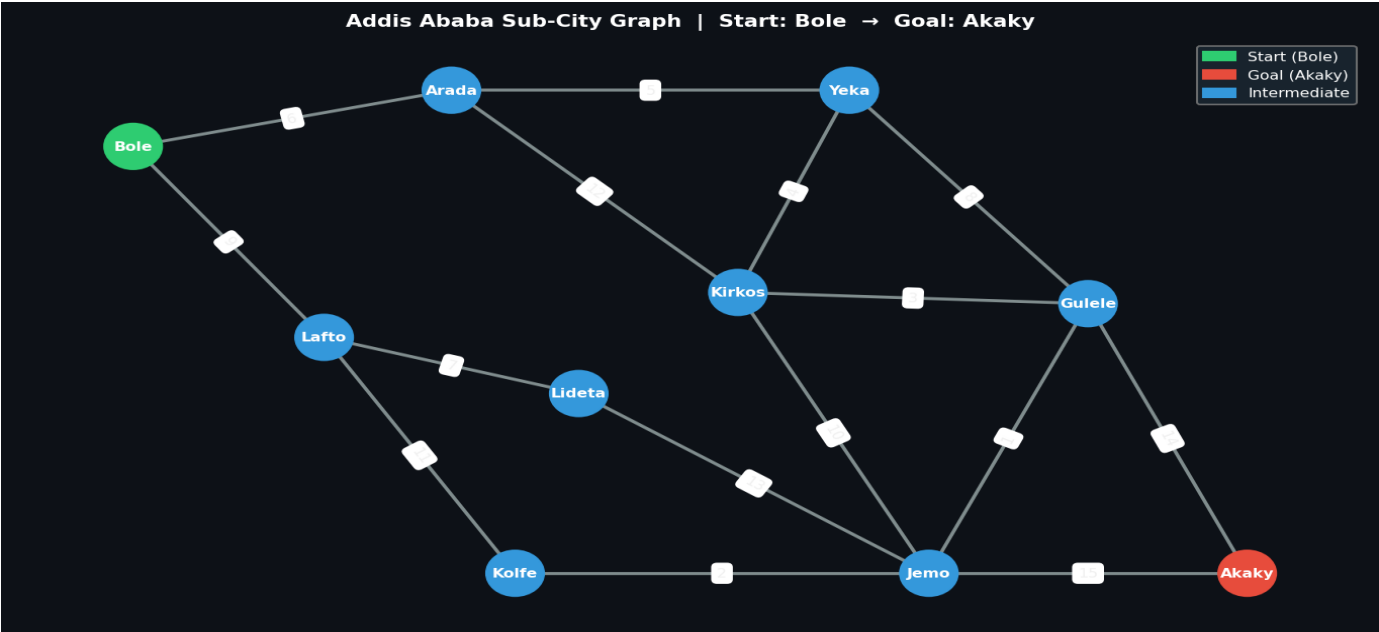


Figure 1: Weighted undirected graph of 10 Addis Ababa sub-cities. Edge labels = travel cost. Optimal path (cost 32): Bole → Arada → Yeka → Kirkos → Gulele → Akaky.

From	To	Cost
Bole	Arada	6
Bole	Lafto	9
Arada	Yeka	5
Arada	Kirkos	12
Lafto	Lideta	7
Lafto	Kolfe	11
Yeka	Kirkos	4
Yeka	Gulele	8
Kirkos	Gulele	3
Kirkos	Jemo	10
Lideta	Jemo	13
Kolfe	Jemo	2
Gulele	Jemo	1
Gulele	Akaky	14
Jemo	Akaky	15

Table 1: Edge weights (left) Table 2: Heuristic values — Original / Good / Misleading (right)

Node	Orig	Good	Mislead
Bole	32	28	10
Arada	26	22	50
Lafto	24	20	5
Yeka	21	18	60
Kirkos	17	15	55
Lideta	28	14	3
Kolfe	17	2	40
Gulele	14	13	50
Jemo	15	14	1
Akaky	0	0	0

Analysis and Comparison

1. Which algorithm reached the goal faster?

GBFS reached the goal faster in terms of nodes expanded. Under the original heuristic it expanded only **5 nodes** versus A*'s 6, and under the good heuristic 6 vs 9. GBFS orders its priority queue purely by $h(n)$, so it commits immediately to the direction that looks closest to the goal and never revisits alternatives. This greedy strategy minimises expansions but sacrifices path quality. A* must evaluate $f(n) = g(n) + h(n)$ for every candidate, forcing it to explore competing routes before it can guarantee the cheapest one — hence more expansions but a better result.

2. Which algorithm produced the optimal solution?

A* produced the optimal solution under both the original and good heuristics, finding Bole → Arada → Yeka → Kirkos → Gulele → Akaky with cost **32** (the true shortest path). GBFS returned cost 37 under both those heuristics. A* is optimal because $g(n)$ tracks the actual cost already paid, and when the heuristic is admissible (never overestimates), the algorithm is guaranteed that the first time it reaches the goal it has done so via the cheapest route. GBFS has no such guarantee — ignoring $g(n)$ means a locally attractive node can lead to a globally expensive path.

Heuristic	Algorithm	Path Found	Cost	Nodes Exp.
Original	GBFS	Bole→Lafto→Kolfe→Jemo→Akaky	37	5
Original	A*	Bole→Arada→Yeka→Kirkos→Gulele→Akaky	32	6
Good	GBFS	Bole→Lafto→Kolfe→Jemo→Akaky	37	6
Good	A*	Bole→Arada→Yeka→Kirkos→Gulele→Akaky	32	9
Misleading	GBFS	Bole→Lafto→Lideta→Jemo→Akaky	44	5
Misleading	A*	Bole→Lafto→Lideta→Jemo→Akaky	44	5

Table 3: Full results summary

Experimental Exploration

Observation 1 — Original Heuristic

GBFS expanded 5 nodes, greedily following Lafto ($h=24$) over Arada ($h=26$), then chaining Kolfe→Jemo because Kolfe's $h=17$ looked promising. Path cost: 37. A* expanded 6 nodes but corrected course — its $g(n)$ accounting revealed the northern corridor costs only 32. This one-node difference in expansions delivers a 5-unit saving in path cost.

Observation 2 — Good Heuristic

The good heuristic sets Kolfe $h=2$, making the southern route look almost free. A* therefore explored 9 nodes — more than with the original — as it thoroughly audited southern nodes before confirming the northern path is truly cheaper. GBFS still chose the southern route (cost 37) without checking. Key insight: a tighter heuristic does not always mean fewer expansions; the interplay between $h(n)$ values and edge weights drives behaviour.

Observation 3 — Misleading Heuristic

Both algorithms were misled. Low h values on Lafto (5) and Lideta (3) made the Lideta detour irresistible, and both found only the costliest path (cost 44) while expanding just 5 nodes. This breaks A*'s optimality guarantee — the heuristic is not admissible, so $f(n)$ no longer bounds the true cost. Low expansion count here signals failure, not efficiency. Heuristic design is the single most critical factor in informed search performance.

```
PS C:\Users\Mekdelawit\AI_GBFS_A-star> python experimental_exploration.py
EXPERIMENTAL EXPLORATION & TESTING
Graph: Addis Ababa sub-cities | Bole -> Akaky
HEURISTIC SET: ORIGINAL

h(n) values:
Bole      h = 32
Arada     h = 26
Lafto     h = 24
Yeka      h = 21
Kirkos    h = 17
Lideta    h = 28
Kolfe     h = 17
Gulele    h = 14
Jemo      h = 15
Akaky     h = 0

[GBFS]
Path Found      : Bole -> Lafto -> Kolfe -> Jemo -> Akaky
Total Path Cost : 37
Nodes Expanded  : 5 (Bole -> Lafto -> Kolfe -> Jemo -> Akaky)

[A*]
Path Found      : Bole -> Arada -> Yeka -> Kirkos -> Gulele -> Akaky
Total Path Cost : 32
Nodes Expanded  : 6 (Bole -> Arada -> Yeka -> Kirkos -> Gulele -> Akaky)
```

Screenshot 1: Execution output — Original heuristic (GBFS & A*)

Execution Output Screenshots



Screenshot 2: Execution output — Good heuristic (GBFS & A*)



Screenshot 3: Execution output — Misleading heuristic (GBFS & A*)



Screenshot 4: Comparison summary table — all heuristics & algorithms